

Assignment - 3

DA6400: Reinforcement Learning

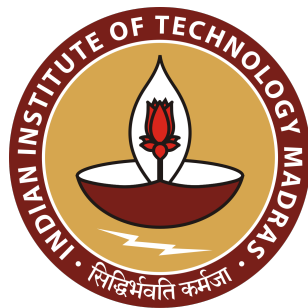
By

Ashmit Sinha and Shahid Mulla

ME21B037 and ME21B122

Course Instructor:

Prof. Balaram Ravindran



Indian Institute of Technology, Madras

March, 2025

Contents

1	Environment & Options Description	1
1.1	Taxi-v3 Environment	1
1.1.1	State Space	1
1.1.2	Action Space	2
1.1.3	Reward Function	2
1.2	Options Description	2
1.2.1	Definition	2
1.2.2	Option Policy	2
1.2.3	Termination Conditions	3
1.2.4	Integrated Action-Option Framework	3
2	SMDP Q-Learning	4
2.1	Algorithm Overview	4
2.2	Pseudocode	4
2.3	Implementation Specifics	5
2.4	Results	6
2.4.1	Training Performance	6
2.4.2	Behavior Analysis	6
2.4.3	Greedy Q-Policy Visualization	7
2.4.4	Option Policy Visualization	8
2.5	Inferences	8

3	Intra-Option Q-Learning	10
3.1	Algorithm Overview	10
3.1.1	Update Equation for Primitive Actions	10
3.1.2	Update Equation for Options	10
3.2	Pseudocode	11
3.3	Implementation Specifics	12
3.4	Results	12
3.4.1	Training Performance	12
3.4.2	Behavior Analysis	13
3.4.3	Greedy Q-Policy Visualization	14
3.4.4	Option Policy Visualization	14
3.5	Inferences	15
4	Alternate Options	16
4.1	Overview	16
4.2	Implementation Specifics	16
4.3	Results: SMDP Q-Learning	16
4.3.1	Training Performance	17
4.3.2	Behavior Analysis	17
4.3.3	Greedy Q-Policy Visualization	18
4.3.4	Option Policy Visualization	18
4.4	Results: Intra-Option Q-Learning	19
4.4.1	Training Performance	19
4.4.2	Behavior Analysis	20
4.4.3	Greedy Q-Policy Visualization	20
4.4.4	Option Policy Visualization	21
4.5	Inferences	21
5	Comparison between SMDP and Intra-Option Q-Learning	22
6	Appendix - (Code Snippets & Github Link)	24

Chapter 1

Environment & Options Description

1.1 TAXI-V3 ENVIRONMENT

The environment used in this assignment is the **Taxi-v3** environment from Gymnasium's classic control suite. A visual representation is shown in Figure 1.1.

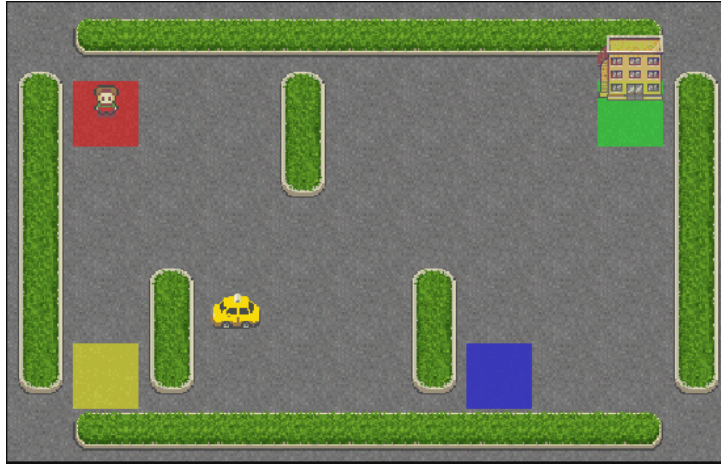


Figure 1.1: Taxi-v3 Environment

1.1.1 State Space

- The environment consists of a 5×5 grid.
- There are four designated landmarks: R(ed), G(reen), Y(ellow), and B(lue).
- A state is encoded as a single integer but can be decoded into a tuple (x, y, p, d) , where:
 - (x, y) : Taxi's current grid position.
 - p : Passenger location
(0: R, 1: G, 2: Y, 3: B, 4: in taxi).
 - d : Passenger destination
(0: R, 1: G, 2: Y, 3: B).
- Total number of discrete states: $25 \times 5 \times 4 = 500$.

1.1.2 Action Space

The agent in the **Taxi-v3** environment can choose from six discrete and deterministic actions at each timestep. These actions are as follows:

- **0 (South)**: Moves the taxi one cell down unless blocked by a wall or boundary.
- **1 (North)**: Move the taxi one cell up, subject to the same movement constraints.
- **2 (East)**: Move the taxi one cell to the right, unless obstructed.
- **3 (West)**: Move the taxi one cell to the left.
- **4 (Pick Up Passenger)**: Attempt to pick up the passenger if the taxi is located at the passenger’s current position.
- **5 (Drop Off Passenger)**: Attempt to drop off the passenger if the taxi is at the destination and the passenger is currently in the taxi.

Movement actions are only successful if they do not violate the environment’s grid constraints or attempt to cross walls. Similarly, **pick** and **drop** actions succeed only under valid conditions.

1.1.3 Reward Function

The reward structure is designed to encourage efficient task completion while penalizing illegal or inefficient actions. The reward function is as follows:

- The agent receives a reward of -1 at every timestep, incentivizing faster completion of the task.
- A reward of $+20$ is given when the passenger is successfully delivered to the correct destination.
- A penalty of -10 is applied if the agent attempts to pick up the passenger from an incorrect location or tries to drop them off at an invalid location.

1.2 OPTIONS DESCRIPTION

1.2.1 Definition

Options are temporally extended actions—predefined behaviors that the agent can choose from, in addition to primitive actions.

- In our implementation, the options are labeled as ‘R’, ‘G’, ‘Y’, and ‘B’.
- Each option represents navigating the taxi to the corresponding colored landmark on the grid.

1.2.2 Option Policy

- Each option maintains a separate policy learned using a tabular Q-learning algorithm.
- Once an option is selected, it controls the taxi’s movement until it reaches its target location.

1.2.3 Termination Conditions

- An option terminates when the taxi reaches the destination associated with that option:
 - Option ‘R’: Ends at $(0, 0)$
 - Option ‘G’: Ends at $(0, 4)$
 - Option ‘Y’: Ends at $(4, 0)$
 - Option ‘B’: Ends at $(4, 3)$

1.2.4 Integrated Action-Option Framework

- At each timestep, the agent chooses among:
 - Primitive actions: `pick`, `put`
 - Learned options: ‘R’, ‘G’, ‘Y’, ‘B’
- Two hierarchical learning algorithms are used:
 - 1-step SMDP Q-learning
 - Intra-Option Q-learning
- The goal is to learn the best meta-policy for task completion (passenger pick-up and delivery) by combining primitive and temporally extended actions.

Chapter 2

SMDP Q-Learning

In this chapter, we implement the Semi-Markov Decision Process (SMDP) Q-learning algorithm, which generalizes standard Q-learning to support temporally extended actions or options with variable durations.

2.1 ALGORITHM OVERVIEW

SMDP Q-learning extends traditional Q-learning by enabling the agent to learn over temporally extended actions, known as options. These options can span multiple time steps, making the learning process more structured and potentially more sample-efficient.

The core idea is that each option, when selected, executes its own policy until it terminates. The update of the Q-values depends on the cumulative reward obtained during the execution of the option and the total duration of that option.

The agent maintains a single Q-table:

- Q_{SMDP} : Stores the Q-values for each state-option pair (s, o) .

Let s be the current state, o the selected option, r the cumulative reward obtained over the option's duration, τ the number of steps taken by the option, and s' the resulting state. The Q-value is updated using the SMDP Q-learning update rule:

$$Q(s, o) \leftarrow Q(s, o) + \alpha \left(r + \gamma^\tau \max_{o'} Q(s', o') - Q(s, o) \right)$$

where:

- α is the learning rate,
- γ is the discount factor,
- τ is the duration (number of steps) taken by the option o ,
- r is the cumulative reward received during the execution of the option,
- s' is the state reached after the option terminates.

2.2 PSEUDOCODE

The following pseudocode outlines the SMDP Q-learning algorithm:

Algorithm 1 SMDP Q-Learning

```
1: Initialize  $Q(s, o)$  arbitrarily
2: for each episode do
3:   Initialize state  $s$ 
4:   while not terminal do
5:     Select option  $o$  using  $\epsilon$ -greedy on  $Q(s, \cdot)$ 
6:     Execute option  $o$  until it terminates
7:     Observe cumulative reward  $r$ , next state  $s'$ , and duration  $\tau$ 
8:     Update:
```

$$Q(s, o) \leftarrow Q(s, o) + \alpha \left(r + \gamma^\tau \max_{o'} Q(s', o') - Q(s, o) \right)$$

```
9:        $s \leftarrow s'$ 
10:   end while
11: end for
```

2.3 IMPLEMENTATION SPECIFICS

The agent's learning process under SMDP Q-learning involves learning the value of options directly based on the total reward they accumulate and their duration.

- **Option Execution:**

- At each decision point, the agent selects one of the available options using an ϵ -greedy policy.
- The selected option is executed until it terminates, during which a sequence of primitive actions may be taken.

- **Reward Accumulation:**

- During the execution of the option, the rewards are accumulated and the number of steps (duration τ) is counted.

- **Q-Value Update:**

- After the option terminates, the SMDP Q-value for the state-option pair is updated using the cumulative reward and the duration of the option, as given by the SMDP Q-learning update rule.

- **Epsilon-Greedy Exploration:**

- An ϵ -greedy policy is used to choose options during training to ensure a balance between exploration and exploitation.

The agent is trained over a series of episodes, with the accumulated episodic reward recorded for performance tracking and evaluation.

2.4 RESULTS

In this section, we present the results obtained from applying Semi-Markov Decision Process (SMDP) Q-Learning to the Taxi-v3 environment. We use a discount factor $\gamma = 0.9$ and a learning rate $\alpha = 0.1$. An ϵ -greedy policy is used for exploration, where ϵ is decayed at every step (starting from $\epsilon = 1$, decaying to a minimum of 0.0001 with decay factor 0.99), encouraging exploration in early episodes and exploitation in later stages.

2.4.1 Training Performance

The agent is trained using SMDP Q-Learning for 10,000 episodes. Figure 4.1 shows the running average of episodic rewards over every 100 episodes during training.

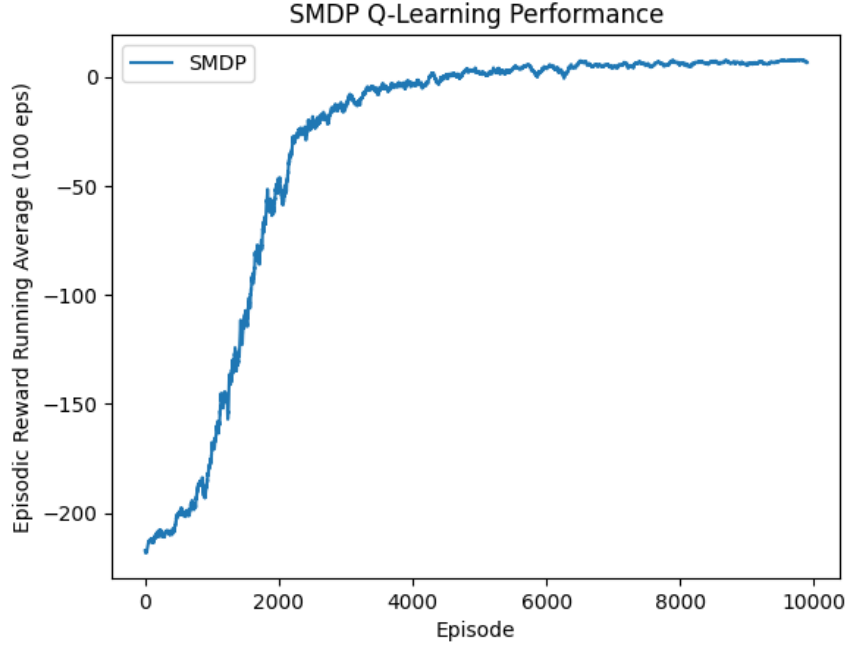


Figure 2.1: Running Average of Episodic Reward vs Number of Episodes (SMDP Q-Learning)

From Figure 4.1, we observe that the agent progressively learns to select actions and options that increase the total episodic reward. This also implies that both the agent’s policy and the internal policies of the options are being learned effectively.

2.4.2 Behavior Analysis

To better understand the policy learned by the agent, we render the final (10,000th) episode, starting from state (3, 4, 2, 0), and visualize the agent’s behavior in Figure 2.2.

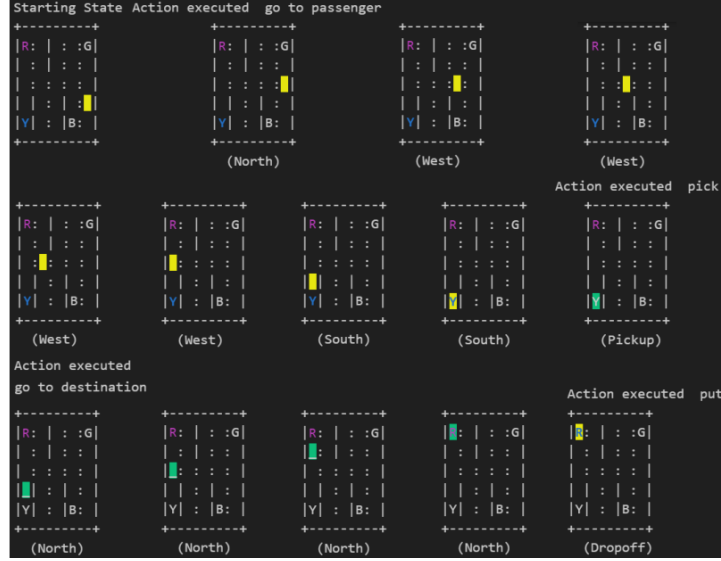


Figure 2.2: Rendering of Episode 10000 with Start State (3, 4, 2, 0) & Total Reward = 8

In this episode, the taxi starts at position (3, 4), the passenger is located at Yellow (2), and the destination is Red (0). The agent executes Option ‘Y’ to reach the Yellow location, then performs the primitive **pick** action. It subsequently executes Option ‘R’ to navigate to the Red drop-off point, and finally performs the **put** action. This sequence reflects optimal behavior.

2.4.3 Greedy Q-Policy Visualization

We are also plotting the greedy Q-values from the agent’s main Q-table for two key phases of the rendered episode: before and after passenger pickup. These are shown in Figure 2.3.

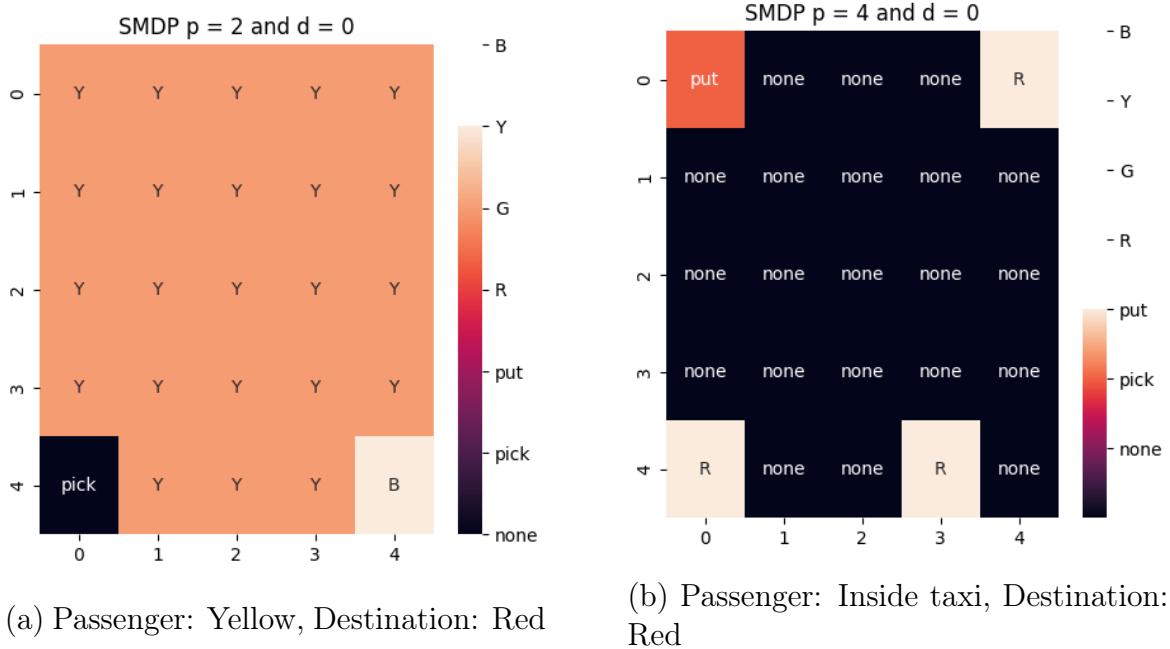


Figure 2.3: Greedy Q-Policy from Main Q-Table for Key States

As observed in Figure 2.3(a), the agent chooses Option ‘Y’ from most states to reach the Yellow location. Once at Yellow, the highest Q-value action becomes `pick`. After pickup, as shown in Figure 2.3(b), Option ‘R’ becomes optimal until the agent reaches the Red drop-off location, where the `put` action is then chosen.

Several states have all Q-values equal to zero, shown as ‘none’ in the visualizations. These are unvisited states. Since SMDP Q-Learning updates only the starting state of an option execution, intermediate states during an option’s execution do not receive updates.

2.4.4 Option Policy Visualization

We also visualize the greedy policies learned by the options used in the rendered episode. These are shown in Figure 2.4.

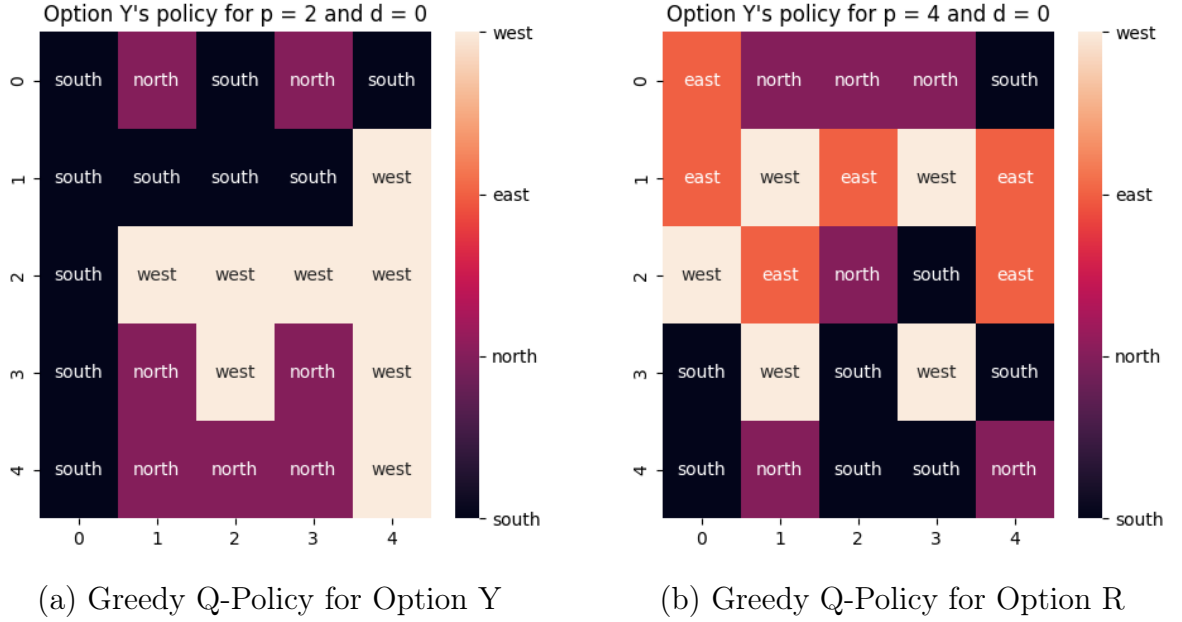


Figure 2.4: Greedy Q-Policies for Options Used in Episode 10000

Figure 2.4 confirms that Option ‘Y’ reliably leads to the Yellow location, and Option ‘R’ guides the taxi efficiently to Red. Both options follow shortest paths while avoiding walls, confirming that the internal option policies are being learned correctly under the SMDP framework.

2.5 INFERENCES

Based on the training outcomes and policy visualizations, we draw the following inferences regarding the behavior of the agent under SMDP Q-Learning:

- The agent successfully learns optimal policies for both high-level option selection and low-level option execution within 10,000 episodes.
- The agent’s action space comprises [`'pick'`, `'put'`, `'R'`, `'G'`, `'B'`, `'Y'`], where `'pick'` and `'put'` are primitive actions, and the remaining are temporally extended options.

- When a primitive action is chosen, standard Q-Learning updates are applied immediately using the observed reward.
- When an option is selected, the agent follows the corresponding option’s policy until termination. No updates are made to the main Q-table during the execution of the option.
- Upon option termination, the Q-value of the starting state-option pair is updated using the cumulative discounted reward collected over the duration of the option. This effectively treats each option as a macro-action with a return-based reward.
- This dual-level learning allows the agent to optimize both the selection of options (in the agent’s main Q-table) and the internal behavior of each option (in its own policy), leading to globally efficient behavior.
- Since each option’s policy is concurrently optimized during training, the paths taken when executing options are near-optimal, resulting in shortest-path navigation to the designated locations.

Chapter 3

Intra-Option Q-Learning

In this chapter, we discuss the Intra-Option Q-Learning (IOQL) algorithm, which is an extension of traditional Q-learning that allows for the learning of policies over options.

3.1 ALGORITHM OVERVIEW

Intra-Option Q-Learning enables the agent to learn over a set of predefined options, with each option corresponding to a higher-level action sequence. The core idea is that the agent can learn policies not only for primitive actions but also for these options, which can significantly speed up learning by focusing on long-term goals.

The agent uses two separate Q-tables:

- The main Q-table: $\mathbf{Q}_{IOQL}$, which stores the values for each state-action pair corresponding to the primitive actions.
- The option-specific Q-tables: $\mathbf{Q}_{\text{option}}$, which store the values for each state-option-action pair corresponding to the options.

Let s denote the current state, a denote the current action, and o denote the current option. The update rules for Intra-Option Q-learning involve both the primitive Q-values and the option Q-values. The update rules are as follows:

3.1.1 Update Equation for Primitive Actions

For each step taken by the agent when it selects a primitive action, the Q-value is updated according to the standard Q-learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_{IOQL} \left(r + \gamma_{IOQL} \max_a Q(s', a) - Q(s, a) \right)$$

where:

- α_{IOQL} is the learning rate for primitive actions,
- γ_{IOQL} is the discount factor,
- r is the immediate reward,
- s' is the next state.

3.1.2 Update Equation for Options

For each step taken by the agent when it selects an option, the Q-value for both the option-specific Q-table and the main Q-table is updated as follows:

$$Q_{\text{option}}(s, o, a) \leftarrow Q_{\text{option}}(s, o, a) + \alpha_{TD} \left(r + \gamma_{\text{policy}} \max_a Q_{\text{option}}(s', o, a) - Q_{\text{option}}(s, o, a) \right)$$

where:

- α_{TD} is the learning rate for options,
- γ_{policy} is the discount factor for option-specific learning.

The Q-value for the main table is also updated:

$$Q(s, o) \leftarrow Q(s, o) + \alpha_{IOQL} (r + \gamma_{IOQL} U - Q(s, o))$$

where U is the utility value of the next state, computed based on whether the goal for the option is achieved:

$$U = (1 - \beta) \cdot Q(s', o) + \beta \cdot \max_a Q(s', a)$$

3.2 PSEUDOCODE

The following pseudocode summarizes the process of training an agent using Intra-Option Q-Learning:

Algorithm 2 Intra-Option Q-Learning

```

1: Initialize  $Q(s, o)$  and  $Q_o(s, a)$  for all options  $o$ 
2: for each episode do
3:   Initialize state  $s$ 
4:   while not terminal do
5:     Choose action  $a$  (primitive or option) using  $\epsilon$ -greedy on  $Q(s, \cdot)$ 
6:     if  $a$  is primitive (pick/drop) then
7:       Take action, observe  $s', r$ 
8:       Update  $Q(s, a)$  with standard Q-learning update
9:     else
10:      Let  $o = a$  be the chosen option
11:      while option  $o$  not terminated do
12:        Choose  $a_o$  using  $\epsilon$ -greedy on  $Q_o(s, \cdot)$ 
13:        Take action  $a_o$ , observe  $s', r$ 
14:        for each option  $o'$  do
15:          Compute termination  $\beta_{o'}(s')$ 
16:          Compute  $U = (1 - \beta_{o'}(s'))Q(s', o') + \beta_{o'}(s') \max_{o''} Q(s', o'')$ 
17:          Update  $Q_{o'}(s, a_o)$  using TD update
18:          Update  $Q(s, o')$  using  $U$ 
19:        end for
20:         $s \leftarrow s'$ 
21:      end while
22:    end if
23:  end while
24: end for

```

3.3 IMPLEMENTATION SPECIFICS

In our implementation, the agent’s learning process is divided into two parts: learning the primitive action policies and learning the option policies.

- **Primitive Actions:**

- The agent first learns to navigate the taxi environment using the primitive actions: South, North, East, West, Pick up, and Drop off.
- The Q-values for these actions are updated according to standard Q-learning using equation (1).

- **Options:**

- Four options—R, G, Y, and B—represent the agent’s decision to drive the taxi to a specific destination on the grid.
- When an option is selected, the agent uses the Q-values stored in the option-specific Q-tables to select an action that will bring it closer to the goal location.
- The option’s policy is updated using the update equation (2), and the main Q-table $\mathbf{Q}_{IOQL}$ is also updated based on the utility value U .

- **Epsilon-Greedy Exploration:**

- Both for primitive actions and options, an ϵ -greedy policy is used to balance exploration and exploitation during training.

- **Goal Checking:**

- The agent checks whether the current state is the goal for a specific option. If the goal is reached, the option terminates, and the agent can choose another option or a primitive action.

The agent is trained for a specified number of episodes, and after each episode, the accumulated reward is recorded for performance evaluation.

3.4 RESULTS

In this section, we present the results obtained by applying Intra-Option Q-Learning (IOQL) on the Taxi-v3 environment. The experiments were performed with a discount factor $\gamma = 0.9$, learning rate $\alpha = 0.1$, and an ϵ -greedy policy for exploration. The ϵ value is decayed at every step (starting from $\epsilon = 1$ and decaying to a minimum of $\epsilon = 0.0001$ with decay factor 0.99) to enable better exploration in early episodes and exploitation towards the end.

3.4.1 Training Performance

We trained the agent using IOQL for 10000 episodes. The average episodic reward over every 100 episodes is plotted against the number of episodes in Figure 3.1.

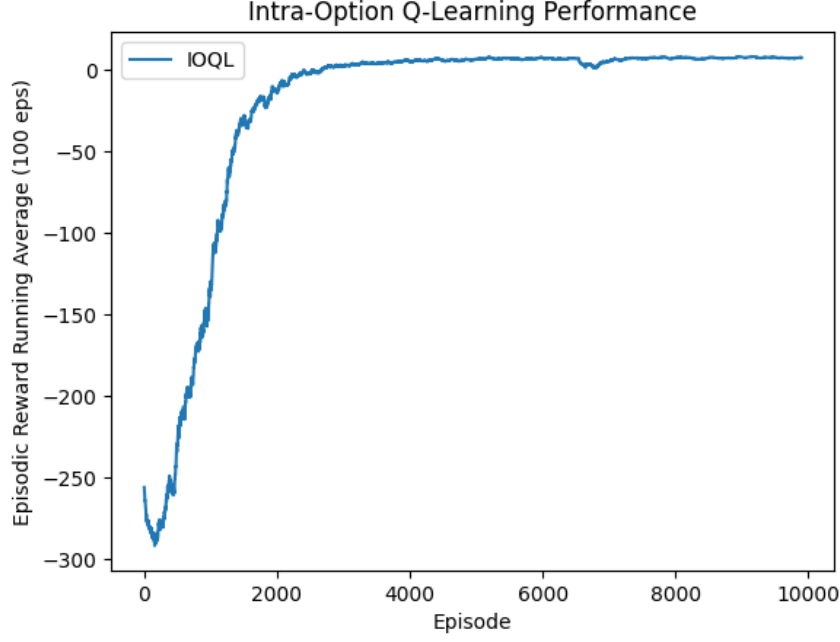


Figure 3.1: Running Average of Episodic Reward vs Number of Episodes

From Figure 3.1, we observe that the agent progressively learns to choose actions and options that maximize the episodic reward. This indicates that both the agent's main Q-table and the policies of the individual options are being learned effectively.

3.4.2 Behavior Analysis

To understand the agent's behavior, we render the final (10000th) episode and visualize the learned policy. The rendered episode, starting from state (2, 0, 3, 1), is shown in Figure 3.2.



Figure 3.2: Rendering of Episode 10000 with Start State (2, 0, 3, 1) & Total Reward = 9

In this episode, the taxi starts at position (2, 0), with the passenger at location Blue (3), and the destination at Green (1). The agent first selects Option ‘B’ and navigates to the Blue square, then executes the primitive ‘pick’ action, followed by Option ‘G’ to navigate to the Green destination, and finally executes the ‘put’ action. This sequence reflects optimal behavior in the learned policy.

3.4.3 Greedy Q-Policy Visualization

We visualize the greedy policy derived from the learned Q-values at key states in the Figure below. Subfigure (a) corresponds to the agent before picking up the passenger, and subfigure (b) corresponds to the post-pickup phase.

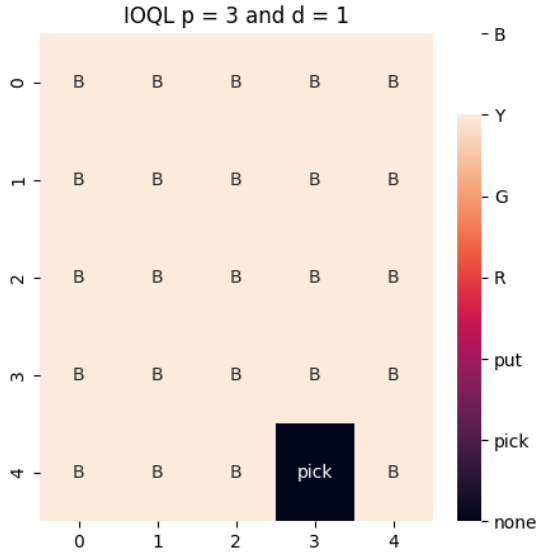


Figure 3.3: Passenger: Blue, Destination: Green

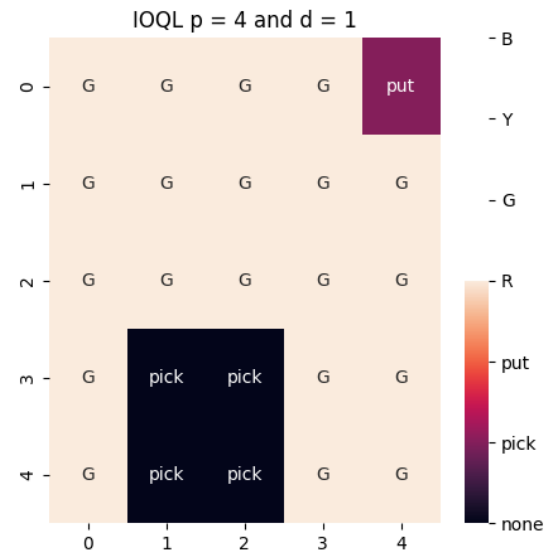


Figure 3.4: Passenger: Inside taxi, Destination: Green

Figure 3.5: Greedy Q-policy from Main Q-table for Key States

As observed, the highest Q-value action before reaching Blue is to execute Option ‘B’. Once at Blue, the highest Q-value corresponds to the ‘pick’ action. After picking up the passenger, the greedy action becomes Option ‘G’, guiding the taxi to the destination. The ‘put’ action then becomes optimal upon reaching the drop location. States with all-zero Q-values are marked as ‘none’, indicating they were never visited during training.

3.4.4 Option Policy Visualization

The learned greedy policies for Option ‘B’ and Option ‘G’, used in the rendered episode, are shown in the Figure below. These policies are derived from the option-specific Q-tables.

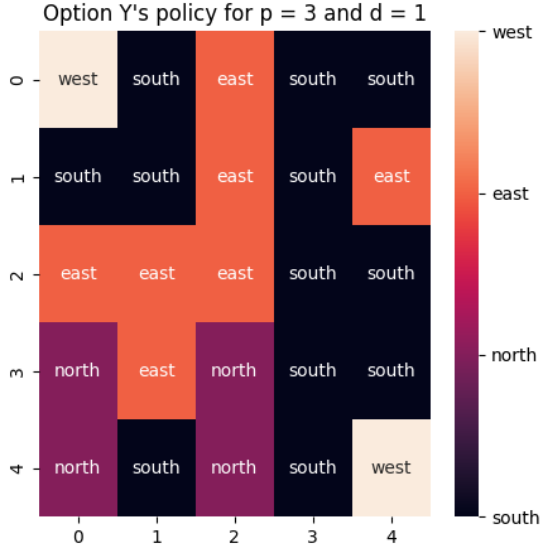


Figure 3.6: Greedy Q-policy for Option B

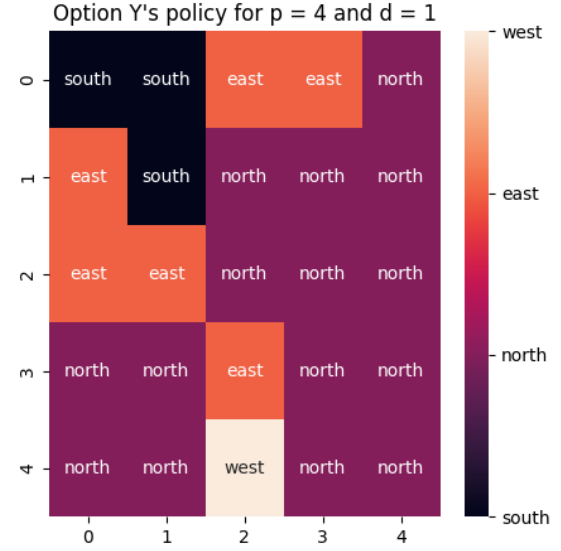


Figure 3.7: Greedy Q-policy for Option G

Figure 3.8: Greedy Q-policies for Options Used in Episode 10000

The option-specific policies clearly show efficient navigation strategies. Option ‘B’ guides the taxi to the Blue location, and Option ‘G’ to the Green location, both taking the shortest paths and avoiding obstacles. This confirms that intra-option learning is functioning as intended.

3.5 INFERENCES

- The agent successfully learns optimal policies for both primitive actions and options within 10,000 episodes using Intra-Option Q-Learning.
- When `pick` or `put` is chosen, their Q-values are updated using the standard Q-learning update rule. Since option policies involve only directional moves (**South**, **North**, **East**, **West**), there is no need to update option-specific Q-values for these primitive actions.
- When the agent selects an option, it follows the corresponding policy until the goal is reached. During this execution, the option’s Q-table is updated at every step, along with the agent’s main Q-table for each consistent option.
- This approach allows the agent to learn from every step taken during an option’s execution by updating not only the chosen option’s Q-values but also those of other relevant options. As a result, Intra-Option Q-Learning makes better use of each experience, learns faster, and converges to optimal behavior more quickly than SMDP Q-Learning.

Chapter 4

Alternate Options

4.1 OVERVIEW

We explore an alternate set of options for the Taxi-v3 environment: `go to passenger` and `go to destination`. Unlike the fixed-location options (R, G, B, Y) used earlier, these options dynamically depend on the passenger’s current location and destination encoded in the state.

- `go to passenger` terminates when the taxi reaches the passenger’s location and the passenger is not yet picked up.
- `go to destination` terminates when the taxi reaches the destination with the passenger onboard.

The agent can choose from `pick`, `put`, and the two new options. The agent policy is learned using SMDP Q-learning, while each option’s policy is trained using Q-learning.

4.2 IMPLEMENTATION SPECIFICS

Each option is implemented via an `Option` class that maintains its own Q-table and termination condition.

- **Policy Learning:** Options use tabular Q-learning with ϵ -greedy action selection.
- **Termination Condition:** Defined using decoded state information to check for reaching the passenger or destination.
- **SMDP Updates:** The agent updates the Q-value of the option upon its termination using the cumulative discounted reward.

4.3 RESULTS: SMDP Q-LEARNING

We train the agent using SMDP Q-learning for 10000 episodes with the following hyperparameters: $\gamma = 0.9$, learning rate $\alpha = 0.1$, and ϵ -greedy action selection for both agent-level decisions and option-level policies. The ϵ value is decayed from 1 to 0.0001 with a decay rate of 0.99 to encourage initial exploration and final exploitation.

4.3.1 Training Performance

We plot the running average of episodic rewards over every 100 episodes in Figure 4.1. The plot demonstrates that the agent successfully learns to select actions and options that maximize the cumulative reward.

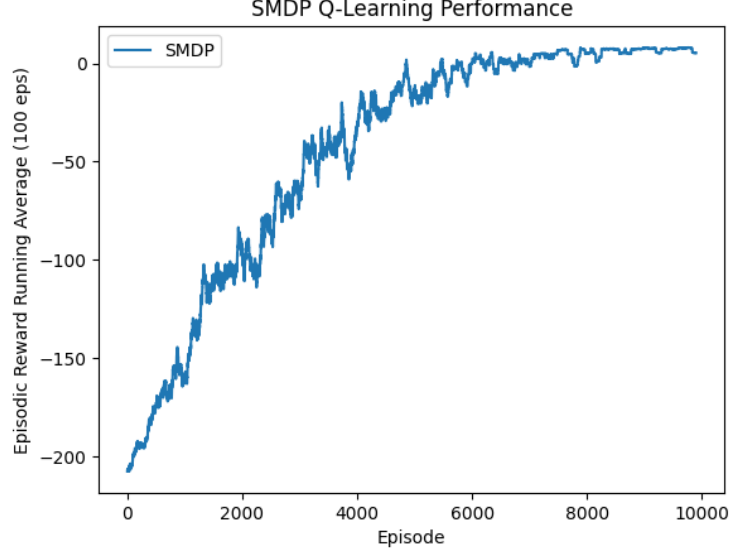


Figure 4.1: Running Average of Episodic Reward vs Number of Episodes

4.3.2 Behavior Analysis

To inspect the learned policy, we render the 10000th episode, starting from the state (1, 1, 3, 2) where the passenger is at location 3 (Blue), the destination is 2 (Yellow), and the taxi is at position (1,1). As shown in Figure 4.2, the taxi first executes the **go to passenger** option to reach Blue, then performs the **pick** action, followed by the **go to destination** option to reach Yellow, and finally executes **put** to complete the task optimally.

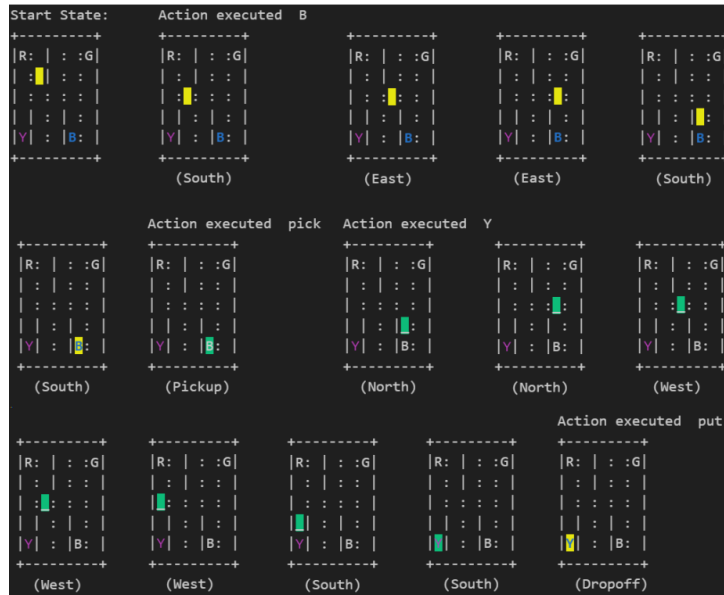


Figure 4.2: Rendering of the environment for start state (1, 1, 3, 2); Total reward: 7

4.3.3 Greedy Q-Policy Visualization

To better understand the learned policies, we visualize the greedy Q-values for two key configurations: before and after passenger pickup. Figure below shows the agent’s greedy Q-values for $(p, d) = (3, 2)$ and $(4, 2)$. Before pickup, the action with the highest Q-value is `go to passenger` in most states, and once at Blue, the highest Q-value corresponds to the `pick` action. After pickup, the policy correctly favors `go to destination` until the agent reaches Yellow, where `put` becomes the optimal action.

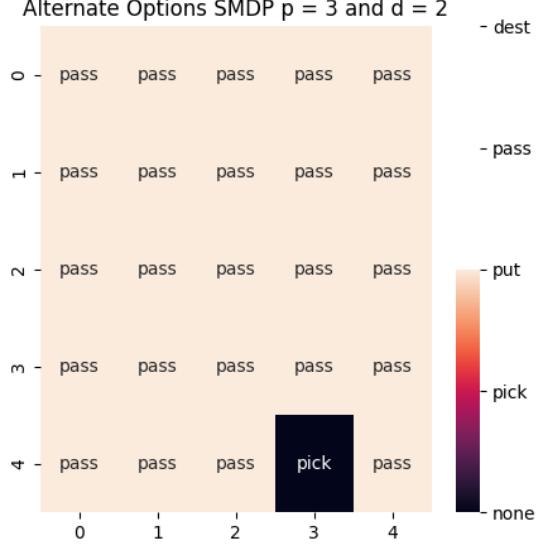


Figure 4.3: Passenger: Blue, Destination: Yellow

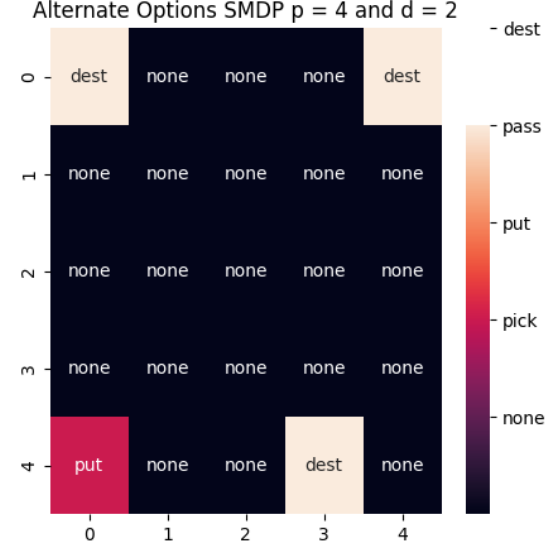


Figure 4.4: Passenger in Taxi, Destination: Yellow

Figure 4.5: Greedy Q policy for the agent before and after passenger pickup

We observe that states which are not visited during training (e.g., intermediate states within an option) exhibit zero Q-values, leading to `none` in the greedy policy plots. This is expected, as SMDP Q-learning only updates the Q-value of the initiating state of an option and not of intermediate states traversed during the option execution.

4.3.4 Option Policy Visualization

We also visualize the internal greedy Q policies of the two options used: `go to passenger` and `go to destination`. As seen in the Figure below, the Q-values correctly guide the taxi along the shortest path to the Blue and Yellow locations, respectively. This confirms that the intra-option policies are being learned effectively during training.



Figure 4.6: Option: Go to Passenger



Figure 4.7: Option: Go to Destination

Figure 4.8: Greedy Q policies for learned options

4.4 RESULTS: INTRA-OPTION Q-LEARNING

We train the agent using Intra-Option Q-learning for 10000 episodes with the following hyperparameters: $\gamma = 0.9$, learning rate $\alpha = 0.1$, and ϵ -greedy action selection for both agent-level decisions and option-level policies. The ϵ value is decayed from 1 to 0.0001 with a decay rate of 0.99 to encourage initial exploration and final exploitation.

4.4.1 Training Performance

We plot the running average of episodic rewards over every 100 episodes in Figure 4.9. The plot demonstrates that the agent successfully learns to choose actions and options that optimize the episodic reward. This also indicates that the option policies are being learned simultaneously with the agent's policy.

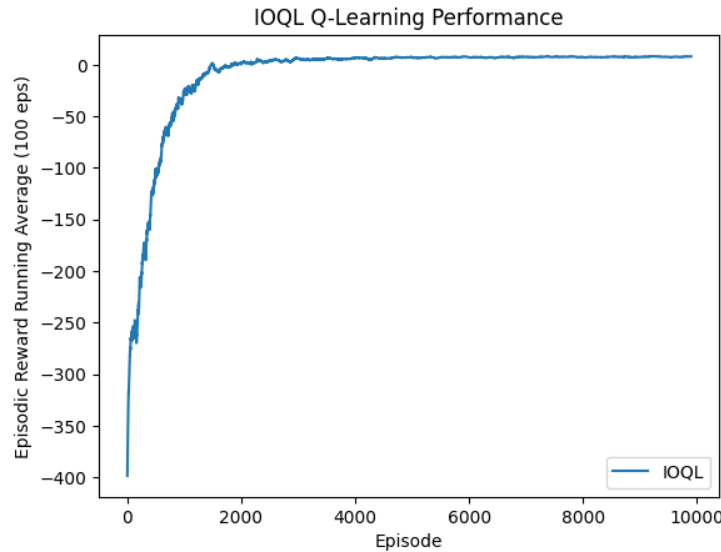


Figure 4.9: Running Average of Episodic Reward vs Number of Episodes

4.4.2 Behavior Analysis

To inspect the learned policy, we render the 10000th episode, starting from the state $(4, 0, 2, 1)$ where the passenger is at location 2 (Yellow), the destination is 1 (Green), and the taxi is at position $(4, 0)$. As shown in Figure 4.10, the taxi first executes the **go to passenger** option to reach Yellow, then performs the **pick** action, followed by the **go to destination** option to reach Green, and finally executes **put** to complete the task optimally.



Figure 4.10: Rendering of the environment for start state $(4, 0, 2, 1)$; Total reward: 11

4.4.3 Greedy Q-Policy Visualization

To better understand the learned policies, we visualize the greedy Q-values for two key configurations: before and after passenger pickup. Figure 4.13 shows the agent's greedy Q-values for $(p, d) = (2, 1)$ and $(4, 1)$. Before pickup, the action with the highest Q-value is **go to passenger** in most states, and once at Yellow, the highest Q-value corresponds to the **pick** action. After pickup, the policy correctly favors **go to destination**, and at the Green location, the **put** action is selected.

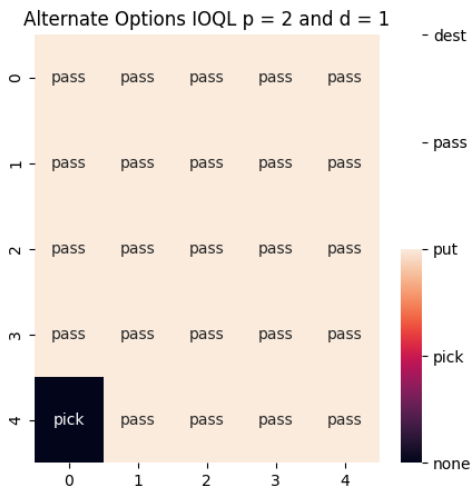


Figure 4.11: Passenger: Yellow, Destination: Green

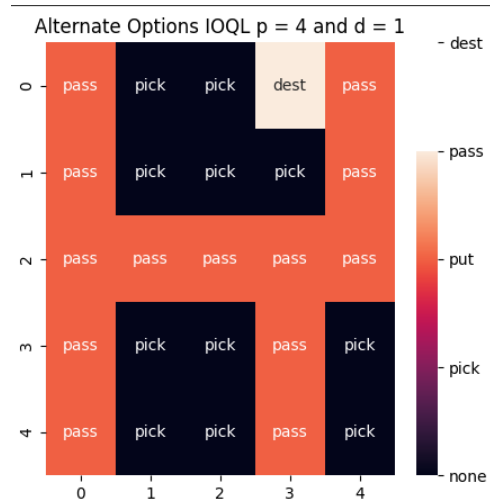


Figure 4.12: Passenger in Taxi, Destination: Green

Figure 4.13: Greedy Q policy for the agent before and after passenger pickup

We observe that some Q-values for the `pick` action appear even after the passenger has been picked up. This is expected in Intra-Option Q-learning, as it updates the Q-values for all states and all options during execution, not just the initiation states. However, since the `go to destination` option dominates the policy after pickup, the agent still navigates optimally and performs the correct `put` action at the destination.

4.4.4 Option Policy Visualization

We also visualize the internal greedy Q policies of the two options used: `go to passenger` and `go to destination`. As seen in Figure 4.16, the Q-values guide the taxi along the shortest paths to the Yellow and Green locations, respectively. This confirms that the intra-option policies are being learned and reinforced effectively throughout the episode.



Figure 4.14: Option: Go to Passenger

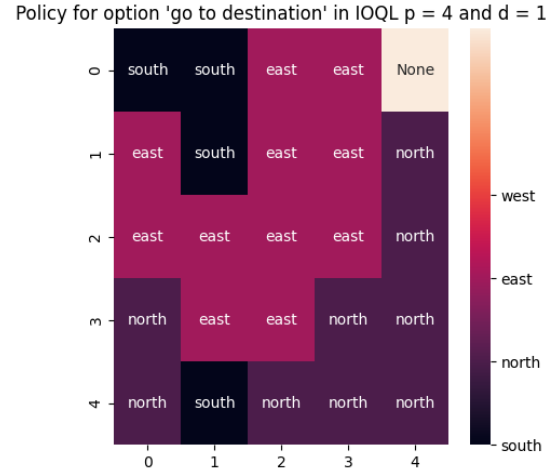


Figure 4.15: Option: Go to Destination

Figure 4.16: Greedy Q policies for learned options

4.5 INFERENCES

- Both SMDP and Intra-Option Q-learning successfully learn optimal policies over 10000 episodes, selecting effectively among `pick`, `put`, `go to passenger`, and `go to destination`.
- Option policies are learned alongside the agent's policy, with `go to passenger` and `go to destination` showing goal-directed behavior.
- Intra-Option Q-learning converges faster than SMDP Q-learning due to more frequent updates across states during option execution.
- With SMDP Q-learning, spatial options (R, G, Y, B) lead to faster learning, while Intra-Option Q-learning performs better with abstract options (`go to passenger`, `go to destination`).

Chapter 5

Comparison between SMDP and Intra-Option Q-Learning

We compare the performance of SMDP Q-learning and Intra-Option Q-learning using both sets of options: spatial options [R, G, Y, B] and abstract options [go to passenger, go to destination]. The comparison is based on learning speed, stability, and update mechanisms.

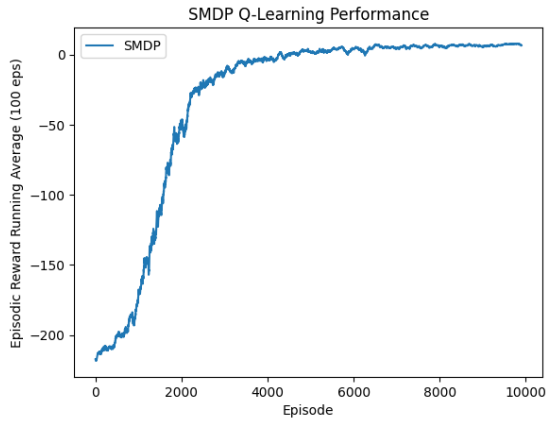


Figure 5.1: SMDP Reward Curve with Original Options

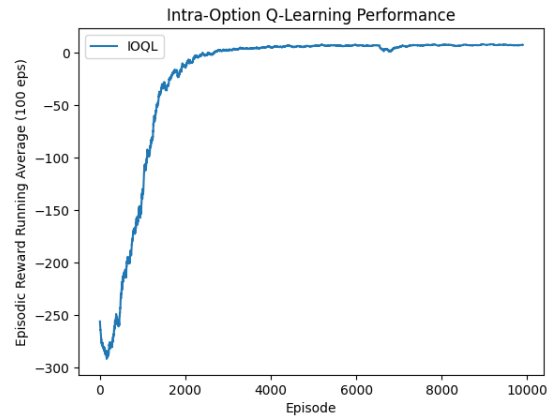


Figure 5.2: IOQL Reward Curve with Original Options

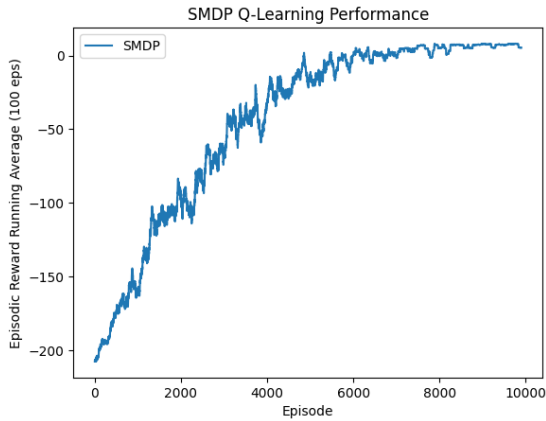


Figure 5.3: SMDP Reward Curve with New Options

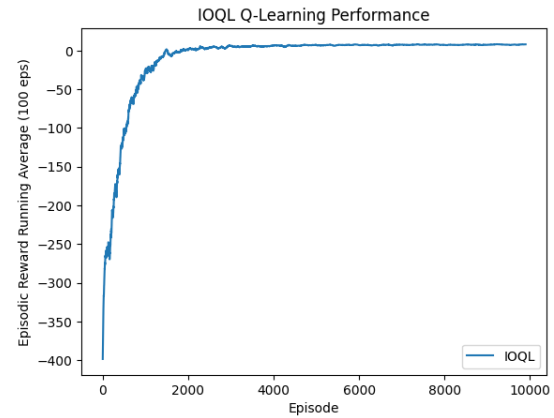


Figure 5.4: IOQL Reward Curve with New Options

- Intra-Option Q-learning learns faster than SMDP Q-learning for both sets of options, as seen in the above figure.
- This is because SMDP Q-learning updates Q-values only at option termination, while Intra-Option Q-learning performs updates at every step for all applicable state-option and state-action pairs.
- The SMDP curves show higher variance compared to the smoother Intra-Option curves, especially for the abstract options.
- Initially, SMDP Q-learning achieves higher rewards, but Intra-Option Q-learning quickly overtakes due to more frequent and distributed updates.

Chapter 6

Appendix - (Code Snippets & Github Link)

```
1 action_option_list = ['down', 'up', 'right', 'left', 'pick', 'put', 'R', 'G', 'Y', 'B']
2
3 goal = {'R': [0, 0], 'G': [0, 4], 'Y': [4, 0], 'B': [4, 3]}
4
5 class Option:
6     def __init__(self, name, state_size, action_size, goal, alpha_td=0.1, gamma=0.9):
7         self.name = name
8         self.goal = goal
9         self.q_table = np.zeros((state_size, action_size))
10        self.alpha_td = alpha_td
11        self.gamma = gamma
12
13    def is_terminal(self, state, env):
14        x, y, _, _ = list(env.unwrapped.decode(state))
15        return [x, y] == self.goal
16
17    def act(self, state, epsilon):
18        if np.random.rand() < epsilon:
19            return np.random.randint(self.q_table.shape[1])
20        else:
21            return np.argmax(self.q_table[state])
22
23    def update(self, state, action, reward, next_state):
24        td_target = reward + self.gamma * np.max(self.q_table[next_state])
25        self.q_table[state, action] += self.alpha_td * (td_target - self.q_table[state, action])
```

Listing 6.1: Option Class and Initialization

```
1 class Agent:
2     def __init__(self, env, action_option_list, goal, alpha_smdp=0.1, alpha_td=0.1, gamma=0.9,
3                 epsilon_start=1.0, epsilon_end=0.0001, epsilon_decay=0.99):
4         self.env = env
5         self.gamma = gamma
6         self.alpha_smdp = alpha_smdp
7         self.epsilon = epsilon_start
8         self.epsilon_end = epsilon_end
9         self.epsilon_decay = epsilon_decay
10        self.action_option_list = action_option_list
11
12        state_size = env.observation_space.n
13        primitive_actions = 2
```

```

14         self.q_smdp = np.zeros((state_size, 6)) # primitive + options
15
16         self.options = {
17             name: Option(name, state_size, env.action_space.n - 2, goal[name],
18 alpha_td, gamma)
19             for name in ['R', 'G', 'Y', 'B']
20         }
21
22     def epsilon_greedy(self, q_values):
23         if np.random.rand() < self.epsilon:
24             return np.random.randint(len(q_values))
25         return np.argmax(q_values)
26
27     def decay_epsilon(self):
28         self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)
29
30     def plot_policy_heatmap(self, q_table, pas=0, des=0, title="SMDP Policy
Heatmap"):
31         grid = np.full((5, 5), -1) # default to -1 (none)
32
33         for state in range(self.env.observation_space.n):
34             x, y, p, d = self.env.unwrapped.decode(state)
35             if p == pas and d == des:
36                 best_action = np.argmax(q_table[state])
37                 grid[x, y] = best_action
38
39         action_ids = [0, 1, 2, 3, 4, 5]
40         labels = {0: 'pick', 1: 'put', 2: 'R', 3: 'G', 4: 'Y', 5: 'B', -1: '
none'}
41         annot_text = np.empty_like(grid, dtype=object)
42
43         for i in range(5):
44             for j in range(5):
45                 annot_text[i, j] = labels[grid[i, j]]
46
47         fig, ax = plt.subplots(figsize=(6, 6))
48         cmap = sns.color_palette("tab10", n_colors=6)
49         heatmap = sns.heatmap(grid.T, ax=ax, annot=annot_text.T, fmt='', cmap=
cmap,
50                                cbar=True, vmin=0, vmax=5, linewidths=0.5,
51                                linecolor='gray')
52
53         ax.set_title(f"{title} (p={pas}, d={des})")
54         cbar = heatmap.collections[0].colorbar
55         cbar.set_ticks(action_ids)
56         cbar.set_ticklabels([labels[i] for i in action_ids])
57
58         ax.set_xlabel("X")
59         ax.set_ylabel("Y")
60         ax.invert_yaxis()
61         plt.show()
62
63     def train_episode(self, verbose=False):
64         state = self.env.reset()[0]
65         done = False
66         total_reward = 0
67
68         if verbose:
69             x, y, p, d = list(self.env.unwrapped.decode(state))
70             print(f"Start State: {x, y, p, d}")
71             print(self.env.render())
72
73         while not done:

```

```

72     action = self.epsilon_greedy(self.q_smdp[state])
73     self.decay_epsilon()
74
75     if verbose:
76         print("Action executed:", self.action_option_list[action + 4])
77
78     if action < 2: # primitive action
79         next_state, reward, done_flag, terminated, _ = self.env.step(
80             action + 4)
81         done = done_flag or terminated
82
83         self.q_smdp[state, action] += self.alpha_smdp * (
84             reward + self.gamma * np.max(self.q_smdp[next_state]) -
85             self.q_smdp[state, action]
86         )
87
88         state = next_state
89         total_reward += reward
90
91     if verbose:
92         print(self.env.render())
93     else: # executing option
94         option_key = self.action_option_list[action + 4]
95         option = self.options[option_key]
96         state_start = state
97         option_done = False
98         option_reward = 0
99         reward_bar = 0
100         tau = 0
101
102         while not option_done:
103             opt_action = option.act(state, self.epsilon)
104             next_state, reward, done_flag, terminated, _ = self.env.
105             step(opt_action)
106             done = done_flag or terminated
107
108             option_done = option.is_terminal(next_state, self.env)
109             option.update(state, opt_action, reward, next_state)
110
111             reward_bar += (self.gamma ** tau) * reward
112             tau += 1
113             state = next_state
114             total_reward += reward
115
116         if verbose:
117             print(self.env.render())
118
119         if done:
120             break
121
122         # SMDP update
123         self.q_smdp[state_start, action] += self.alpha_smdp * (
124             reward_bar + (self.gamma ** tau) * np.max(self.q_smdp[state

```

Listing 6.2: SMDP Q-Learning Agent Class

```

1 class IOQLAgent:
2     def __init__(self, env, action_option_list, goal, alpha_ioql=0.1, alpha_td
    =0.1, gamma_ioql=0.9, gamma_policy=0.9,

```

```

3         epsilon_start=1.0, epsilon_end=0.0001, epsilon_decay=0.99):
4     self.env = env
5     self.gamma_ioql = gamma_ioql
6     self.gamma_policy = gamma_policy
7     self.alpha_ioql = alpha_ioql
8     self.alpha_td = alpha_td
9     self.epsilon = epsilon_start
10    self.epsilon_opt = epsilon_start
11    self.epsilon_end = epsilon_end
12    self.epsilon_decay = epsilon_decay
13    self.action_option_list = action_option_list
14
15    state_size = env.observation_space.n
16    primitive_actions = 2
17
18    self.q_ioql = np.zeros((state_size, 6)) # primitive + options
19
20    self.options = {
21        name: Option(name, state_size, env.action_space.n - 2, goal[name],
22        alpha_td, gamma_policy)
23        for name in ['R', 'G', 'Y', 'B']
24    }
25
26    def epsilon_greedy(self, q_values, epsilon):
27        if np.random.rand() < epsilon:
28            return np.random.randint(len(q_values))
29        return np.argmax(q_values)
30
31    def decay_epsilons(self):
32        self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)
33        self.epsilon_opt = max(self.epsilon_end, self.epsilon_opt * self.
34        epsilon_decay)
35
36    def train_episode(self, verbose=False):
37        reset_result = self.env.reset()
38        state = reset_result[0] if isinstance(reset_result, tuple) else
39        reset_result
40        done = False
41        total_reward = 0
42
43        if verbose:
44            x, y, p, d = list(self.env.unwrapped.decode(state))
45            print(f"Start State: {x, y, p, d}")
46            print(self.env.render())
47
48        while not done:
49            self.decay_epsilons()
50            action = self.epsilon_greedy(self.q_ioql[state], self.epsilon)
51
52            if verbose:
53                print("Action executed:", self.action_option_list[action + 4])
54
55            if action < 2: # primitive action
56                next_state, reward, done_flag, terminated, _ = self.env.step(
57                action + 4)
58                done = done_flag or terminated
59
60                self.q_ioql[state, action] += self.alpha_ioql * (
61                    reward + self.gamma_ioql * np.max(self.q_ioql[next_state])
62                    - self.q_ioql[state, action]
63                )
64
65                state = next_state

```

```

61         total_reward += reward
62
63         if verbose:
64             print(self.env.render())
65
66         else: # option
67             option_key = self.action_option_list[action + 4]
68             option = self.options[option_key]
69             init_state = state
70             option_done = False
71
72             while not option_done:
73                 opt_action = self.epsilon_greedy(option.q_table[state],
74 self.epsilon_opt)
75                 next_state, reward, done_flag, terminated, _ = self.env.
76 step(opt_action)
77                 done = done_flag or terminated
78
79                 x, y, _, _ = self.env.unwrapped.decode(next_state)
80                 option_done = [x, y] == option.goal
81
82                 for idx in [2, 3, 4, 5]:
83                     opt_name = self.action_option_list[idx + 4]
84                     opt = self.options[opt_name]
85
86                     # intra-option Q update
87                     opt.q_table[state, opt_action] += self.alpha_td * (
88                         reward + self.gamma_policy * np.max(opt.q_table[
89 next_state]) - opt.q_table[state, opt_action]
90                     )
91
92                     if [x, y] == opt.goal:
93                         self.q_ioql[state, idx] += self.alpha_ioql * (
94                             reward + self.gamma_ioql * np.max(self.q_ioql[
95 next_state]) - self.q_ioql[state, idx]
96                         )
97                     else:
98                         self.q_ioql[state, idx] += self.alpha_ioql * (
99                             reward + self.gamma_ioql * self.q_ioql[
100 next_state, idx] - self.q_ioql[state, idx]
101                         )
102
103                 state = next_state
104                 total_reward += reward
105
106                 if verbose:
107                     print(self.env.render())
108                 if done:
109                     break
110
111             return total_reward

```

Listing 6.3: Intra-Option Q-Learning Agent Class

```

1 new_action_option_list = ['down', 'up', 'right', 'left', 'pick', 'put', '
2     passenger', 'destination']
3
4 class Option:
5     def __init__(self, name, state_size, action_size, alpha_td=0.1, gamma=0.9):
6         self.name = name
7         self.q_table = np.zeros((state_size, action_size))
8         self.alpha_td = alpha_td
9         self.gamma = gamma

```

```

9
10     def is_terminal(self, state, env):
11         x, y, p, d = list(env.unwrapped.decode(state))
12         if self.name == 'passenger':
13             return p != 4 and (x, y) == env.unwrapped.locs[p] # taxi at
passenger location but passenger not in car
14         elif self.name == 'destination':
15             return p == 4 and (x, y) == env.unwrapped.locs[d] # taxi at
destination but passenger still in car
16         return False
17
18     def act(self, state, epsilon):
19         if np.random.rand() < epsilon:
20             return np.random.randint(self.q_table.shape[1])
21         else:
22             return np.argmax(self.q_table[state])
23
24     def update(self, state, action, reward, next_state):
25         td_target = reward + self.gamma * np.max(self.q_table[next_state])
26         self.q_table[state, action] += self.alpha_td * (td_target - self.
q_table[state, action])

```

Listing 6.4: Option Class with Alternate Option Set

GitHub Link: <https://github.com/DA6400-RL-JanMay2025/programming-assignment-03-me21b122-me21b037>